

```

;          Write X86/64 ALP to convert 4-digit Hex number
into its equivalent BCD number

; and 5-digit BCD number into its equivalent HEX number.

; Provide a menu for user choice (a) HEX to BCD (b) BCD to
HEX (c) EXIT

; Display appropriate prompts and results. Use 64-bit
registers wherever required.

```

```

; Macro for write syscall

```

```

%macro write 2

```

```

    mov rax, 1      ; syscall number for write

    mov rdi, 1      ; file descriptor (stdout)

    mov rsi, %1     ; buffer address

    mov rdx, %2     ; buffer length

    syscall

```

```

%endmacro

```

```

; Macro for read syscall

```

```

%macro read 2

```

```

    mov rax, 0      ; syscall number for read

    mov rdi, 0      ; file descriptor (stdin)

    mov rsi, %1     ; buffer address

    mov rdx, %2     ; buffer length

    syscall

```

```

%endmacro

```

```

section .data

```

```

    dispMsg db "Write X86/64 ALP to convert 4-digit
Hex number into its equivalent BCD number and 5- digit
BCD number into its equivalent HEX number. Make your
program user friendly to accept the choice from user for: (a)
HEX to BCD b) BCD to HEX (c) EXIT. Display proper strings to
prompt the user while accepting the input and displaying the
result. (Wherever necessary, use 64-bit registers).",10,
"Name:- Sumaanyu",10,"Roll:- 7256",10,"Date of
Performance:- 17/03/2025",10

```

```

    dispMsglen equ $-dispMsg

```

```

    msg db "-----MENU-----", 10

```

```

    db "    1. Hex to BCD ", 10

```

```

    db "    2. BCD to Hex ", 10

```

```

    db "    3. Exit ", 10

```

```

    db "Enter your choice : "

```

```

    msglen equ $-msg

```

```

    msg1 db "Number : "

```

```

    len equ $-msg1

```

```

    endl db 10      ; newline character

```

```

section .bss

```

```

    num resd 1      ; reserved for integer

```

```

    choice resb 2    ; user choice input

```

```

    a_hex resb 5     ; input hex string

```

```

    a_bcd resb 6     ; input BCD string

```

```

    buffer resb 5    ; buffer to store converted BCD
string

```

```

    temp resb 4      ; temp buffer to store converted
hex string

```

```

section .text

```

```

    global _start

```

```

_start :

```

```

    write dispMsg, dispMsglen    ; display initial
description

```

```

main_menu :

```

```

    write msg, msglen           ; display menu

```

```

    read choice, 2              ; read user choice

```

```

    cmp byte[choice], "1"      ; check for "1" (Hex to
BCD)

```

```

    je case1

```

```

    cmp byte[choice], "2"      ; check for "2" (BCD to
Hex)

```

```

    je case2

```

```

    cmp byte[choice], "3"      ; check for "3" (Exit)

```

	je case3		stack	push dx	; push remainder (digit) on stack
				loop convert	
input	jmp main_menu	; repeat menu on invalid input			
				mov rcx, 5	
case1 :				mov rsi, buffer	; point to output buffer
	write msg1, len	; prompt for hex input			
	read a_hex, 5	; read hex input (4-digit max)	print_bcd :		
	call hex_bcd	; call hex to bcd conversion		pop ax	; get digit from stack
				add ax, 30h	; convert to ASCII
	jmp main_menu	; return to menu		mov [rsi], ax	; store in buffer
				inc rsi	; next position
case2 :				loop print_bcd	
	write msg1, len	; prompt for bcd input			
	read a_bcd, 6	; read bcd input (5-digit max)		write buffer, 5	; display final BCD string
	call bcd_hex	; call bcd to hex conversion		write endl, 1	; newline
	jmp main_menu	; return to menu		ret	
case3 :				; -----	
	mov rax, 60	; syscall for exit		; Procedure to convert BCD number to hexadecimal	
	mov rdi, 0			; -----	
	syscall		bcd_hex :		
				mov rsi, a_bcd	; point to input BCD string
; -----				mov rcx, 5	; loop for 5 digits
; Procedure to convert hexadecimal number to BCD				mov bx, 0AH	; base 10 multiplier
; -----				mov rax, 0	; clear RAX
hex_bcd:					
	call ascii_hex	; convert input string to binary value (in BX)	b_convert :		
				mul bx	; multiply RAX by 10
	mov ax, bx	; move result to AX		mov dl, [rsi]	; get digit
	mov bx, 0AH	; divisor = 10 (decimal)		sub dl, 30h	; convert ASCII to integer
digit	mov rcx, 5	; loop 5 times to extract each digit		add al, dl	; add to accumulator
				inc rsi	; next digit
convert :				loop b_convert	
	mov rdx, 0	; clear remainder			
	div bx	; divide AX by 10		mov bx, ax	; move final result to BX
				call display	; call display procedure

```

ret

mov al, bl
and al, 0Fh      ; mask lower nibble
cmp al, 9
jbe add30h      ; if <= 9, add 30h
add al, 7h      ; else add 7h for 'A'-'F'

; -----
; Procedure to convert ASCII hex string to binary number
; -----

ascii_hex:
    mov rbx, 0      ; clear RBX
    mov rsi, a_hex   ; point to input string
    mov rcx, 4      ; process 4 characters

    next :
        rol bx, 4      ; make space for next hex digit
        mov al, [rsi]   ; load character
        cmp al, '9'
        jbe sub30h     ; if <= '9', just subtract 30h
        sub al, 7h      ; for 'A'-'F', adjust value

    sub30h :
        sub al, 30h     ; convert ASCII to hex
        add bl, al      ; add digit to result
        inc rsi         ; next character
        loop next
        ret

; -----
; Procedure to convert binary hex to ASCII and display
; -----

display :
    mov rsi, temp      ; point to temp output
    buffer

    mov rcx, 4      ; 4 hex digits to display

    next1 :
        rol bx, 4      ; rotate left to extract highest
        nibble

```